[1부] 머신러닝
7장 앙상블 학습과 랜덤 포레스트

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 7장: 앙상블 학습과 랜덤 포레스트

7.1 투표 기반 분류기
7.2 배깅과 페이스팅
7.3 랜덤 패치와 랜덤 서브스페이스
7.4 랜덤 포레스트
7.5 부스팅
7.6 스태킹

7장 앙상블 학습과 랜덤 포레스트

배깅, 부스팅, 스태킹 등 가장 인기 있는 앙상블 방법 학습

7.1 투표 기반 분류기(1)

- 앙상블 학습
  - 일련의 예측기(즉, 분류나 회귀 모델)로부터 예측을 수집하면 가장 좋은 모델 하나보다 더 좋은 예측을 얻을 수 있을 것임
  - 랜덤 포레스트는 결정 트리의 앙상블
- 투표 기반 분류기
  - 정확도가 80%인 분류기 여러 개를 훈련시켰다고 가정: 로지스틱 회귀 분류기, SVM 분류기, 랜덤 포레스트 분류기, K-최근접 이웃 분류기 등

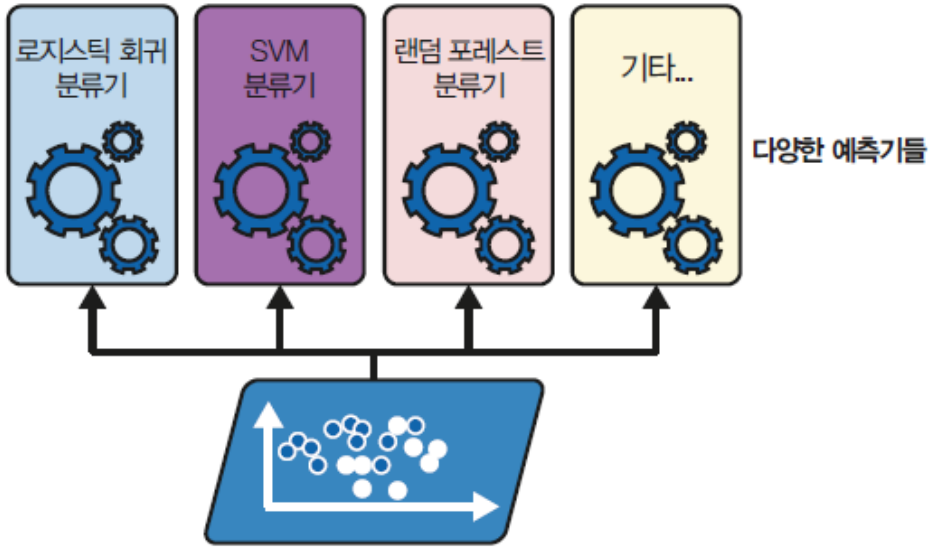

그림 7-1 여러 분류기 훈련시키기

7.1 투표 기반 분류기(2)

- 직접 투표(hard voting) 분류기
  - 강한 학습기
  - 약한 학습기

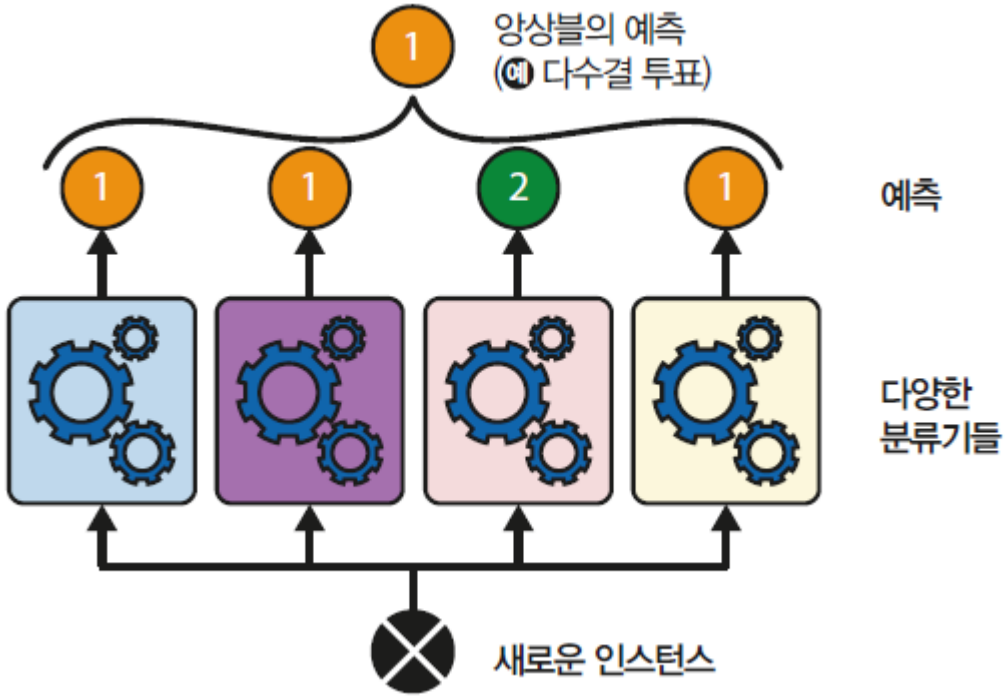

그림 7-2 직접 투표 분류기의 예측

7.1 투표 기반 분류기(3)

- 큰 수의 법칙(law of large numbers)

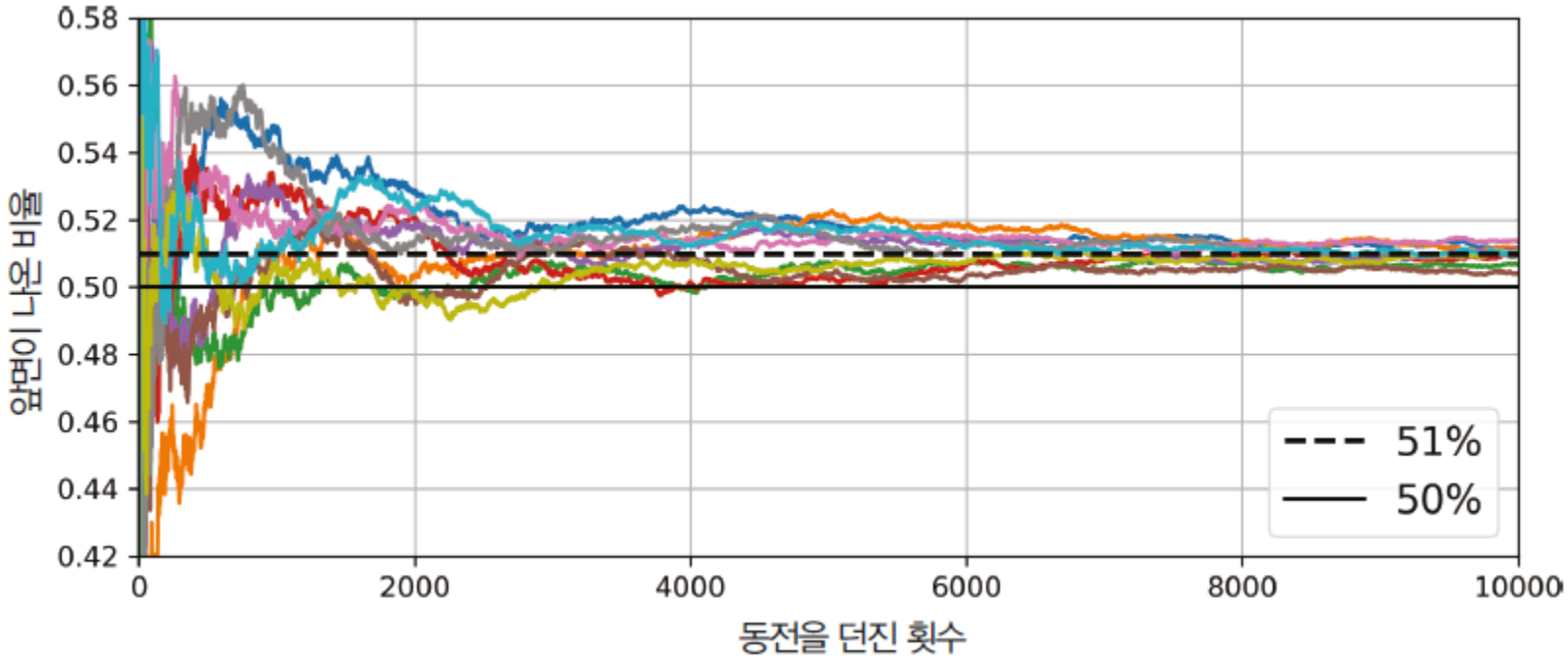

그림 7-3 큰 수의 법칙

7.1 투표 기반 분류기(4)

- moons 데이터셋을 로드하고 훈련 세트와 테스트 세트로 분할한 다음 세 가지 다양한 분류기로 구성된 투표 기반 분류기를 생성하고 훈련
  - VotingClassifier 클래스 사용

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)
```

7.1 투표 기반 분류기(5)

- 테스트 세트에서 훈련된 각 분류기의 정확도 확인

```
>>> for name, clf in voting_clf.named_estimators_.items():
...     print(name, "=", clf.score(X_test, y_test))
...
lr = 0.864
rf = 0.896
svc = 0.896
```

- 투표 기반 분류기의 predict() 메서드를 호출하여 직접 투표를 수행

```
>>> voting_clf.predict(X_test[:1])
array([1])
>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]
```

- 테스트 세트에서 투표 기반 분류기의 성능

```
>>> voting_clf.score(X_test, y_test)
0.912
```

7.1 투표 기반 분류기(6)

- 간접 투표(soft voting)
  - 모든 분류기가 클래스의 확률을 예측할 수 있으면(즉, predict_proba() 메서드가 있으면) 개별 분류기의 예측을 평균 내어 확률이 가장 높은 클래스를 예측할 수 있음
  - 투표 기반 분류기의 voting 매개변수를 "soft"로 바꾸고 모든 분류기가 클래스의 확률을 추정

```
>>> voting_clf.voting = "soft"
>>> voting_clf.named_estimators["svc"].probability = True
>>> voting_clf.fit(X_train, y_train)
>>> voting_clf.score(X_test, y_test)
0.92
```

7.2 배깅과 페이스팅(1)

- 배깅(bagging, bootstrap aggregating의 줄임말)
  - 훈련 세트에서 중복을 허용하여 샘플링하는 방식
- 페이스팅(pasting)
  - 중복을 허용하지 않고 샘플링하는 방식

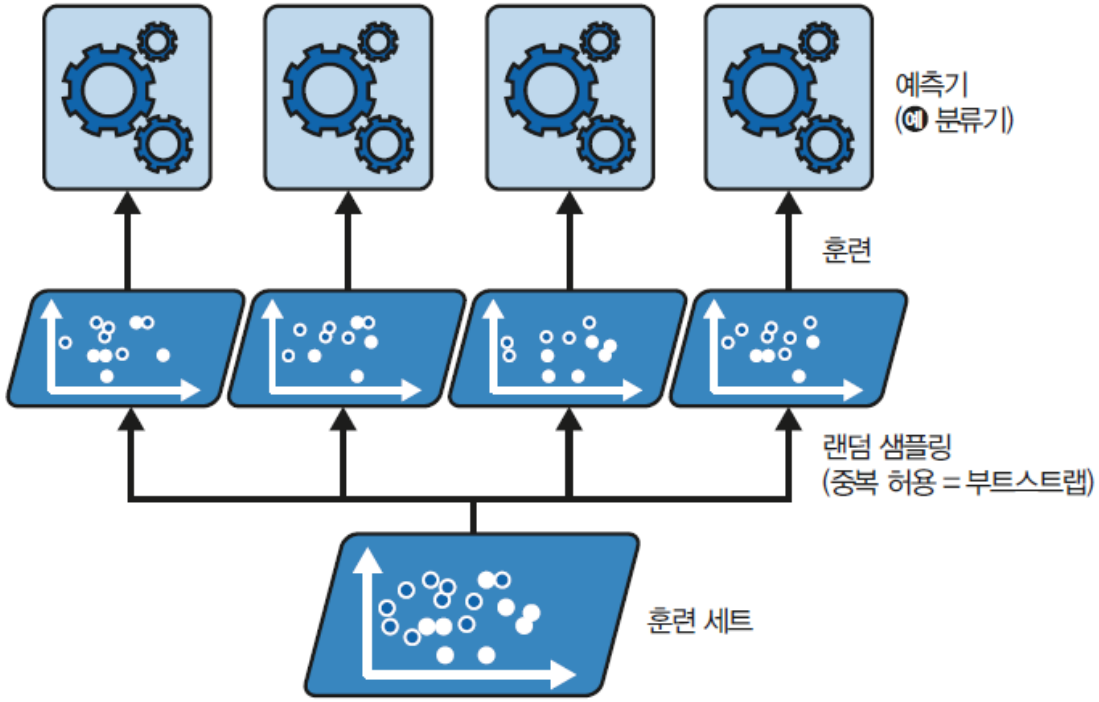

그림 7-4 배깅과 페이스팅은 훈련 세트에서 랜덤으로 샘플링하여 여러 개의 예측기를 훈련

7.2 배깅과 페이스팅(2)

7.2.1 사이킷런의 배깅과 페이스팅

- BaggingClassifier(회귀의 경우에는 BaggingRegressor)

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                            max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```

7.2 배깅과 페이스팅(3)

- 단일 결정 트리의 결정 경계와 500개의 트리를 사용한 배깅 앙상블(이전 코드로 만든 모델)의 결정 경계를 비교

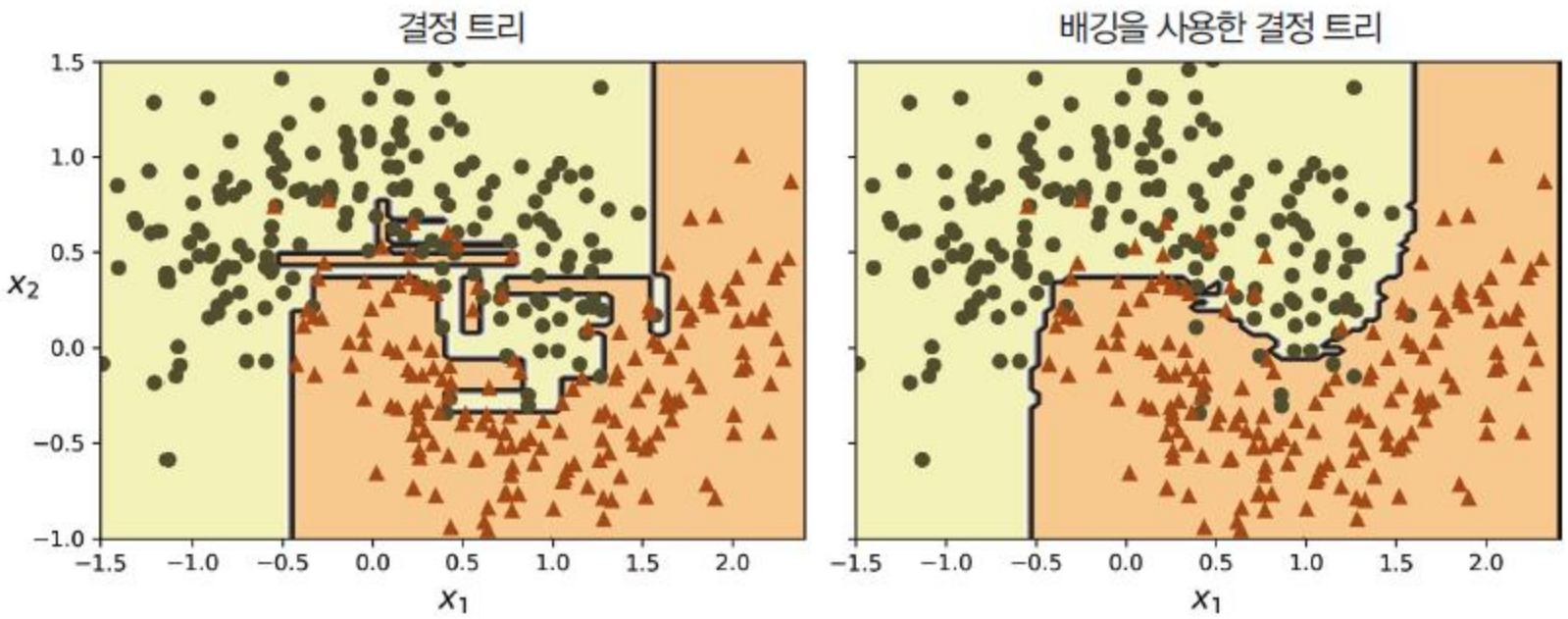

그림 7-5 단일 결정 트리(왼쪽)와 500개의 트리로 만든 배깅 앙상블(오른쪽) 비교

7.2 배깅과 페이스팅(4)

7.2.2 OOB 평가

- BaggingClassifier는 기본값으로 중복을 허용하여(bootstrap=True) 훈련 세트의 크기만큼인 m개 샘플을 선택
  - 이는 평균적으로 각 예측기에 훈련 샘플의 63% 정도만 샘플링된다는 것을 의미
  - OOB(out-of-bag) 샘플 - 선택되지 않은 나머지 37%의 샘플

- oob_score=True로 지정하면 훈련이 끝난 후 자동으로 OOB 평가를 수행

```
>>> bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
...                             oob_score=True, n_jobs=-1, random_state=42)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.896
```

7.2 배깅과 페이스팅(5)

- OOB 평가 결과 확인

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.91200000000000003
```

- OOB 샘플에 대한 결정 함수의 값도 oob_decision_function_ 변수에서 확인

```
>>> bag_clf.oob_decision_function_[:3] # 처음 3개의 샘플에 대한 확률
array([[0.32352941, 0.67647059],
       [0.3375    , 0.6625 ],
       [1.        , 0. ]])
```

7.3 랜덤 패치와 랜덤 서브스페이스

- 랜덤 패치 방식(random patches method)
  - 훈련 특성과 샘플을 모두 샘플링
- 랜덤 서브스페이스 방식(random subspaces method)
  - 훈련 샘플을 모두 사용(bootstrap=False이고 max_samples=1.0으로 설정)
  - 특성을 샘플링(bootstrap_features=True 그리고/또는 max_features는 1.0보다 작게 설정)
- 특성 샘플링은 더 다양한 예측기를 만들며 편향을 늘리는 대신 분산을 낮춤

7.4 랜덤 포레스트(1)

- 500개의 트리로 이뤄진 랜덤 포레스트 분류기를 가능한 모든 CPU 코어에서 훈련시키는 코드

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                 n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

- 랜덤 포레스트 알고리즘은 트리의 노드를 분할할 때 전체 특성 중에서 최선의 특성을 찾는 대신 랜덤으로 선택한 특성 후보 중에서 최적의 특성을 찾는 식으로 무작위성을 더 주입
  - 기본적으로 $\sqrt{n}$개의 특성을 선택

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42)
```

7.4 랜덤 포레스트(2)

7.4.1 엑스트라 트리

- 익스트림 랜덤 트리(extremely randomized tree) 앙상블(또는 줄여서 엑스트라 트리(extra-tree))
  - 극단적으로 랜덤한 트리의 랜덤 포레스트
  - 사이킷런의 ExtraTreesClassifier를 사용

7.4 랜덤 포레스트(3)

7.4.2 특성 중요도

- 랜덤 포레스트의 또 다른 장점은 특성의 상대적 중요도를 측정하기 쉽다는 점
- 훈련이 끝난 뒤 특성마다 자동으로 이 점수를 계산하고 중요도의 전체 합이 1이 되도록 결괏값을 정규화
  - 이 값은 feature_importances_ 변수에 저장되어 있음
- iris 데이터셋에 RandomForestClassifier를 훈련시키고 각 특성의 중요도를 출력

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
...
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

7.4 랜덤 포레스트(4)

- MNIST 데이터셋에 랜덤 포레스트 분류기를 훈련시키고 각 픽셀의 중요도를 시각화

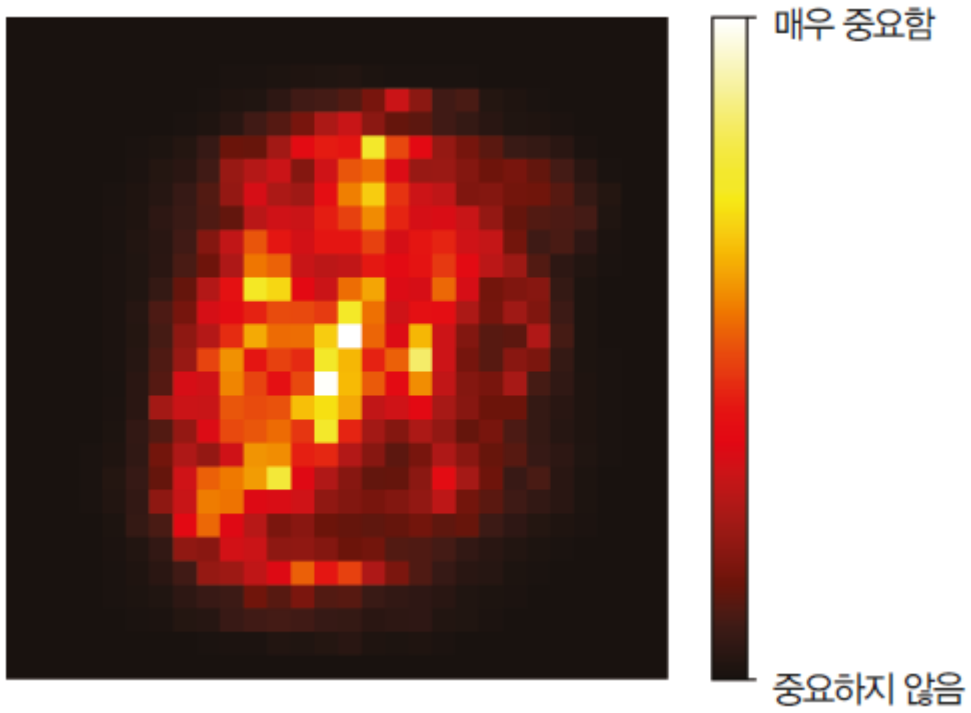

그림 7-6 (랜덤 포레스트 분류기에서 얻은) MNIST 픽셀 중요도

7.5 부스팅(1)

- 부스팅(boosting)
  - 원래는 가설 부스팅(hypothesis boosting)
  - 약한 학습기를 여러 개 연결하여 강한 학습기를 만드는 앙상블 방법
  - 가장 인기 있는 부스팅 방법은 AdaBoost(adaptive boosting, '에이다부스트')와 그레이디언트 부스팅

7.5 부스팅(2)

7.5.1 AdaBoost

- 이전 모델이 과소적합했던 훈련 샘플의 가중치를 더 높임

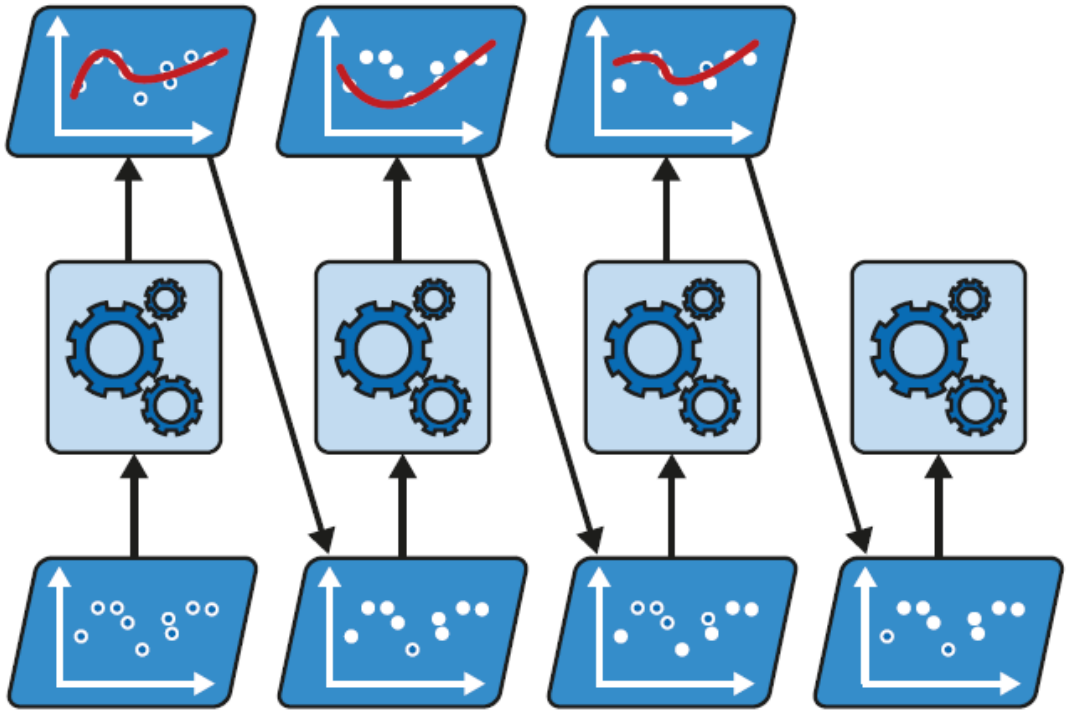

그림 7-7 샘플의 가중치를 업데이트하면서 순차적으로 학습하는 AdaBoost

7.5 부스팅(3)

- moons 데이터셋에 훈련시킨 다섯 개의 연속된 예측기의 결정 경계
  - 이 모델은 규제를 강하게 한 RBF 커널 SVM 분류기

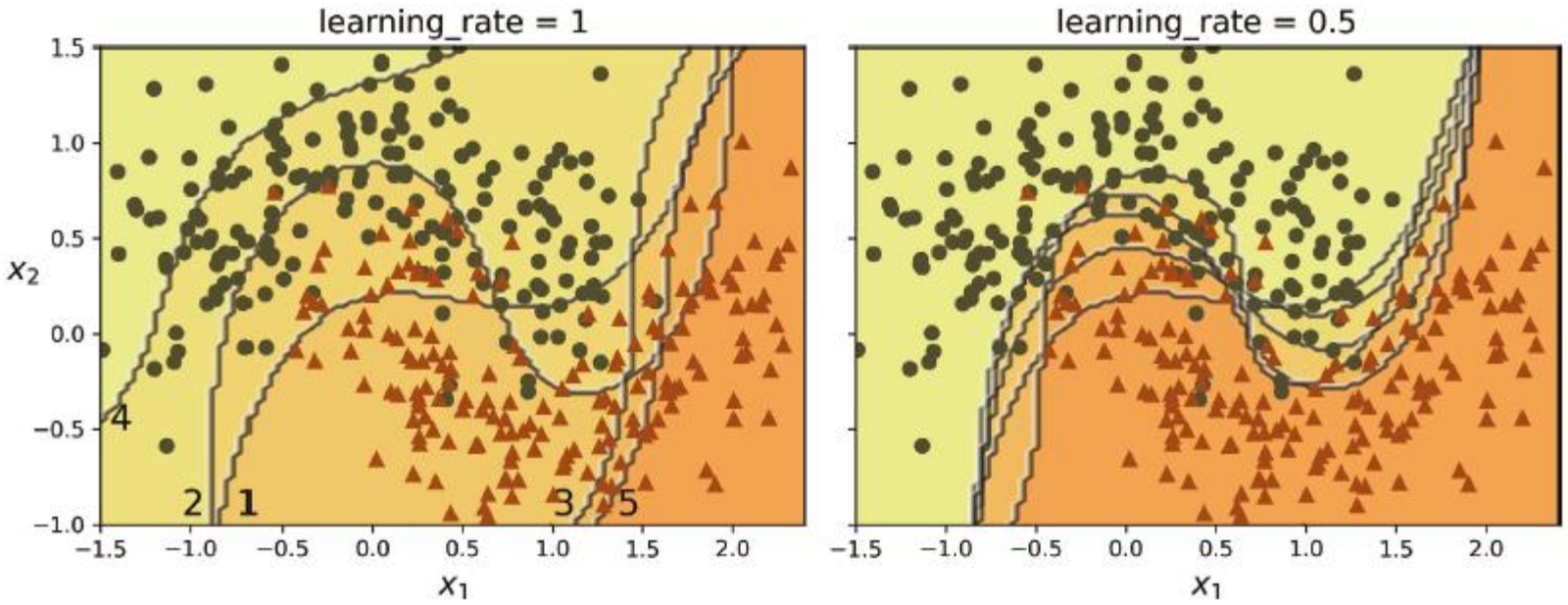

그림 7-8 연속된 예측기의 결정 경계

7.5 부스팅(4)

- 가중치가 적용된 훈련 세트의 전반적인 정확도에 따라 예측기마다 다른 가중치가 적용

식 7-1 j번째 예측기의 가중치가 적용된 오류율

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

여기서 $\hat{y}_j^{(i)}$는 i번째 샘플에 대한 j번째 예측기의 예측

- 예측기가 정확할수록 가중치는 더 높아짐

식 7-2 예측기 가중치

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

7.5 부스팅(5)

- AdaBoost 알고리즘이 [식 7-3]을 사용해 샘플의 가중치를 업데이트
  - 즉, 잘못 분류된 샘플의 가중치가 높아짐

식 7-3 가중치 업데이트 규칙

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \hat{y}_j^{(i)} = y^{(i)}\text{일 때} \\ w^{(i)} \exp(\alpha_j) & \hat{y}_j^{(i)} \neq y^{(i)}\text{일 때} \end{cases} \quad \text{여기서 } i = 1, 2, \cdots, m$$

- 모든 샘플의 가중치를 정규화합니다(즉, $\Sigma_{i=1}^{m} w^{(i)}$로 나눔)
- 마지막으로 새 예측기가 업데이트된 가중치를 사용해 훈련되고 전체 과정이 반복
- 가중치 합이 가장 큰 클래스가 예측 결과

식 7-4 AdaBoost 예측

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x})=k}}^{N} \alpha_j \quad \text{여기서 } N\text{은 예측기 수}$$

7.5 부스팅(6)

- AdaBoostClassifier를 사용하여 200개의 아주 얕은 결정 트리를 기반으로 하는 AdaBoost 분류기를 훈련
  - 결정 트리는 max_depth=1

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
```

7.5 부스팅(7)

7.5.2 그레이디언트 부스팅

- 그레이디언트 트리 부스팅(gradient tree boosting)
  그레이디언트 부스티드 회귀 트리(gradient boosted regression tree, GBRT)

- 2차 방정식으로 잡음이 섞인 데이터셋을 생성하고 DecisionTreeRegressor를 학습

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) # y = 3x² + 가우스_잡음
tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

7.5 부스팅(8)

- 첫 번째 예측기에서 생긴 잔여 오차에 두 번째 DecisionTreeRegressor를 훈련

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)
```

- 두 번째 예측기가 만든 잔여 오차에 세 번째 회귀 모델을 훈련

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)
```

- 모든 트리의 예측을 더함

```
>>> X_new = np.array([[-0.4], [0.], [0.5]])
>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
array([0.49484029, 0.04021166, 0.75026781])
```

7.5 부스팅(9)

- GradientBoostingRegressor를 사용하여 GBRT 앙상블을 훈련

```
from sklearn.ensemble import GradientBoostingRegressor
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                 learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

7.5 부스팅(10)

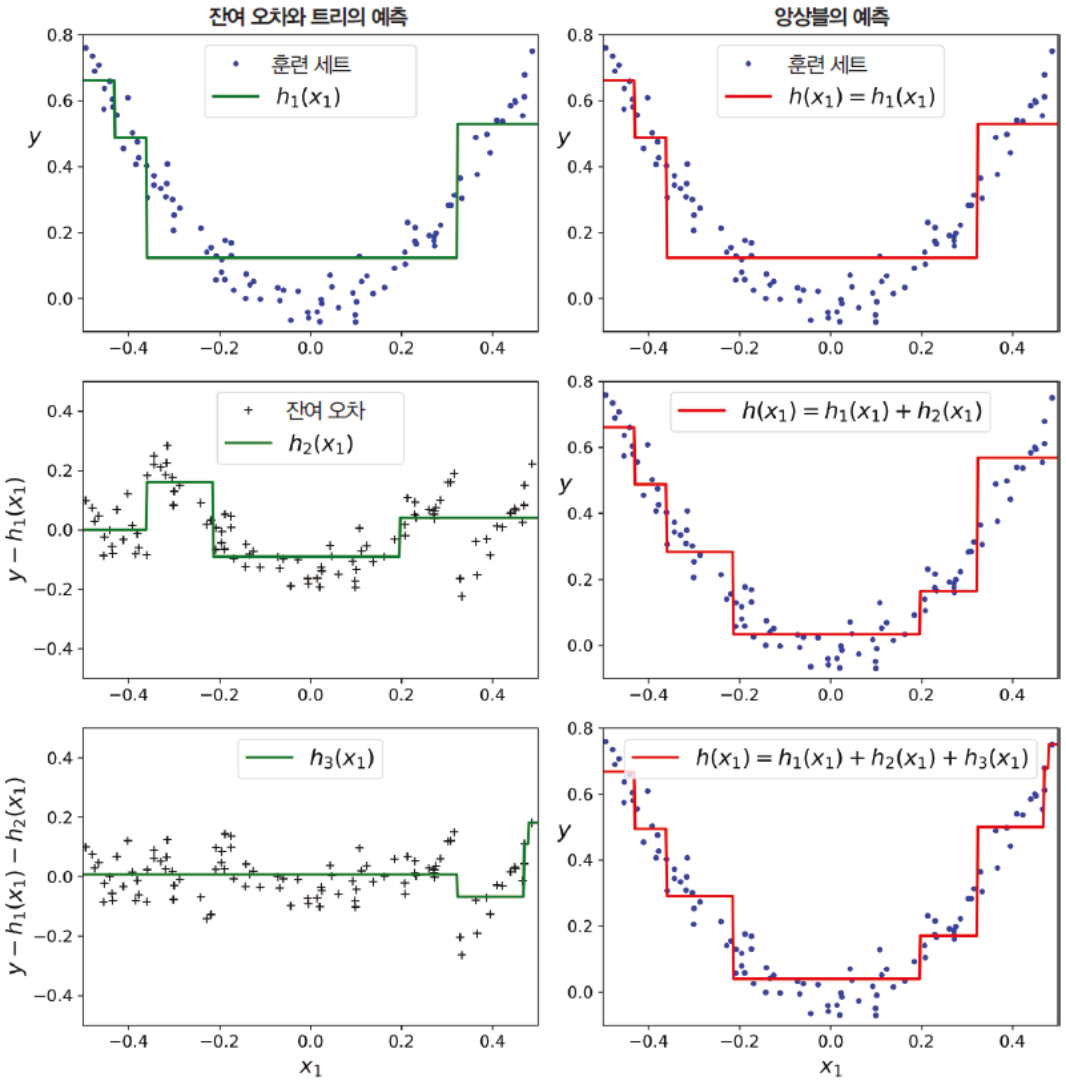

그림 7-9 이 그레이디언트 부스팅 그래프에서 첫 번째 예측기(왼쪽 위)가 평소처럼 훈련. 그다음 연이은 예측기(왼쪽 중간, 왼쪽 아래)가 이전의 예측기의 잔여 오차에서 훈련. 오른쪽 열은 만들어진 앙상블의 예측

7.5 부스팅(11)

- 축소(shrinkage)
  - learning_rate 매개변수가 각 트리의 기여도를 조절
  - 이를 0.05처럼 낮게 설정하면 앙상블을 훈련 세트에 학습시키기 위해 많은 트리가 필요하지만 예측의 성능은 좋아짐

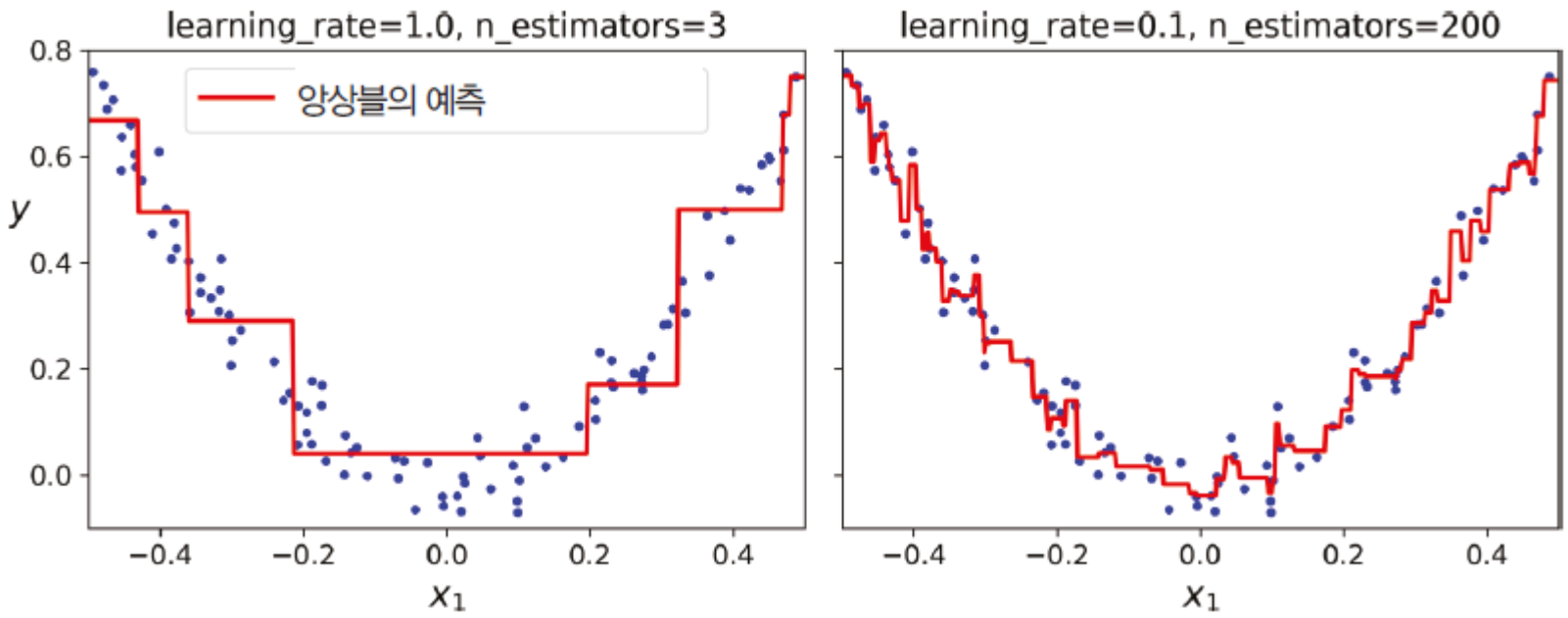

그림 7-10 예측기가 부족한 경우(왼쪽)와 충분한 경우(오른쪽)의 GBRT 앙상블

7.5 부스팅(12)

- n_iter_no_change 하이퍼파라미터를 정숫값(예: 10)으로 설정하면 훈련 중에 마지막 10개의 트리가 도움이 되지 않는 경우 GradientBoostingRegressor가 트리 추가를 자동으로 중지

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2, learning_rate=0.05, n_estimators=500,
    n_iter_no_change=10, random_state=42)
gbrt_best.fit(X, y)
```

- 학습된 앙상블의 실제 추정기 개수는 조기 종료 덕분에 훨씬 적음

```
>>> gbrt_best.n_estimators_
92
```

- 확률적 그레이디언트 부스팅(stochastic gradient boosting)

7.5 부스팅(13)

7.5.3 히스토그램 기반 그레이디언트 부스팅

- 히스토그램 기반 그레이디언트 부스팅(histogram-based gradient boosting, HGB)
  - HistGradientBoostingRegressor와 HistGradientBoostingClassifier
    - 인스턴스 수가 10,000개보다 많으면 조기 종료가 자동으로 활성화
      early_stopping 매개변수를 True 또는 False로 설정하여 조기 종료를 항상 켜거나 끌 수 있음
    - subsample 매개변수가 지원되지 않음
    - n_estimators 매개변수가 max_iter로 바뀌었음
    - 조정할 수 있는 결정 트리 하이퍼파라미터는 max_leaf_nodes, min_samples_leaf, max_depth뿐임

7.5 부스팅(14)

- HGB 클래스는 범주형 특성과 누락된 값을 지원
  - 범주형 특성은 0 ~ max_bins 사이의 정수로 표현(OrdinalEncoder를 사용)
- 캘리포니아 주택 데이터셋에 대한 전체 파이프라인을 구축하고 훈련

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
                            remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)
hgb_reg.fit(housing, housing_labels)
```

7.6 스태킹(1)

- 블렌더(blender) 또는 메타 학습기(meta learner)

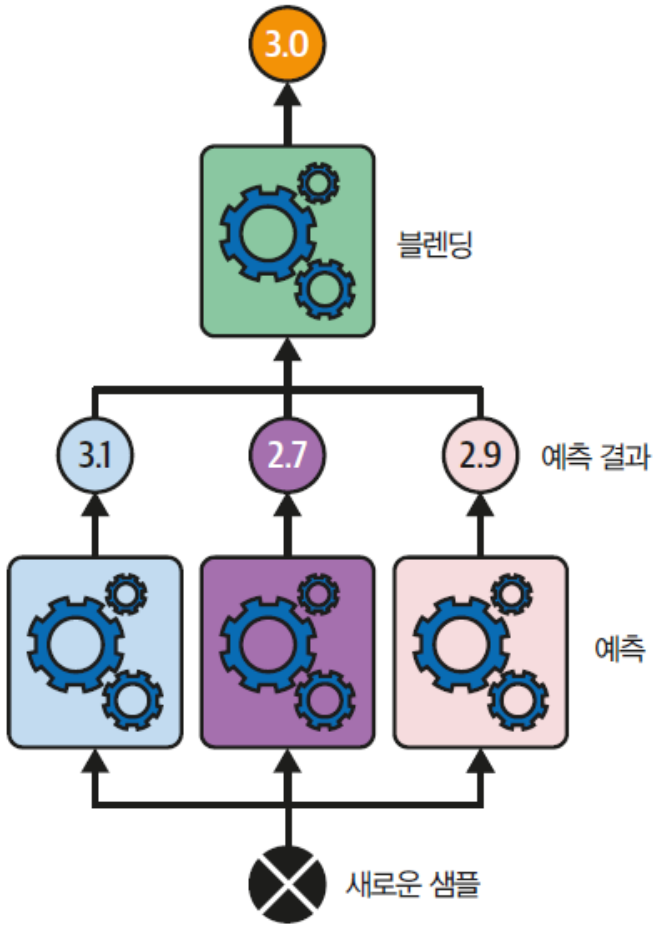

그림 7-11 블렌딩 예측기를 사용한 예측 취합

7.6 스태킹(2)

- 블렌더가 학습되면 기본 예측기는 전체 원본 훈련 세트로 마지막에 한 번 더 재훈련

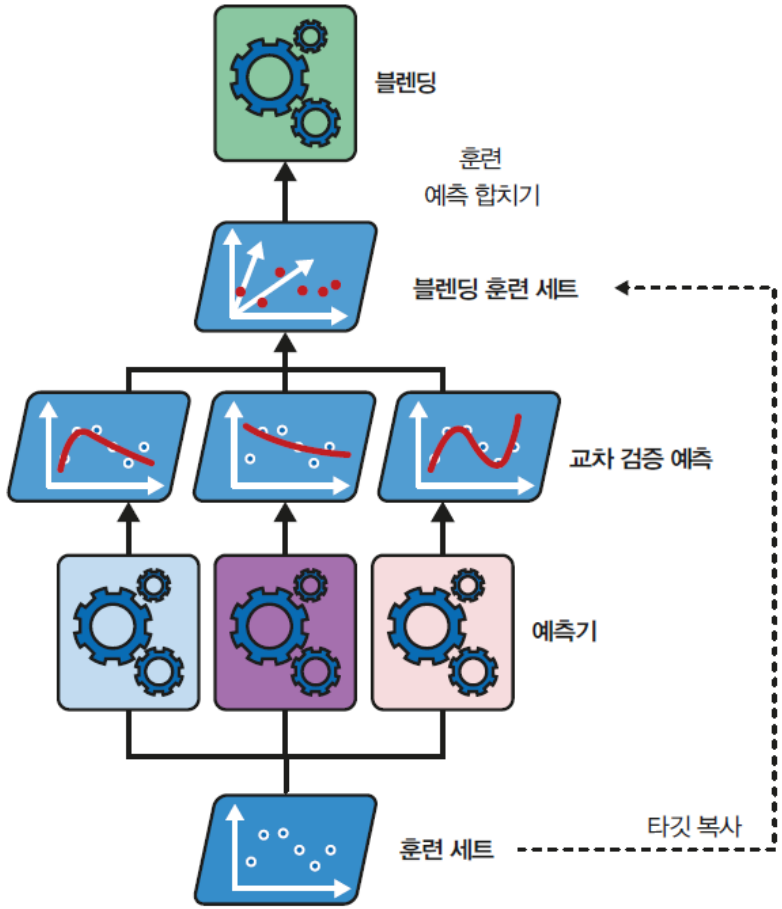

그림 7-12 스태킹 앙상블에서 블렌더 훈련하기

7.6 스태킹(3)

- 여러 가지 블렌더(선형 회귀를 사용하는 블렌더, 랜덤 포레스트 회귀를 사용하는 블렌더 등)를 훈련하여 전체 블렌더 계층을 얻은 다음 그 위에 다른 블렌더를 추가하여 최종 예측을 생성

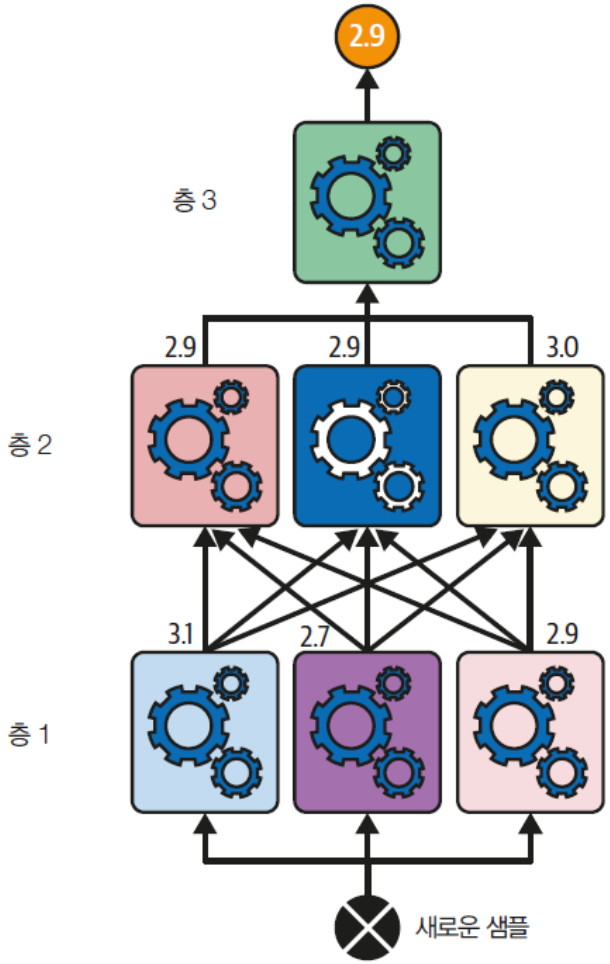

그림 7-13 다층 스태킹 앙상블에서의 예측

7.6 스태킹(4)

- 스태킹 앙상블을 위한 StackingClassifier와 StackingRegressor 클래스

```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # 교차 검증 폴드 개수
)
stacking_clf.fit(X_train, y_train)
```

연습문제(1)

1. 정확히 같은 훈련 데이터로 다섯 개의 다른 모델을 훈련시켜서 모두 95% 정확도를 얻었다면 이 모델들을 연결하여 더 좋은 결과를 얻을 수 있을까? 가능하다면 어떻게 해야 할까? 그렇지 않다면 왜일까?
2. 직접 투표와 간접 투표 분류기 사이의 차이점은 무엇일까?
3. 배깅 앙상블의 훈련을 여러 대의 서버에 분산시켜 속도를 높일 수 있을까요? 페이스팅 앙상블, 부스팅 앙상블, 랜덤 포레스트, 스태킹 앙상블의 경우는 어떨까?
4. oob 평가의 장점은 무엇인가?
5. 무엇이 엑스트라 트리를 일반 랜덤 포레스트보다 더 무작위하게 만드나? 추가적인 무작위성이 어떻게 도움이 될까? 엑스트라 트리는 일반 랜덤 포레스트보다 느릴까요, 빠를까?
6. AdaBoost 앙상블이 훈련 데이터에 과소적합되었다면 어떤 매개변수를 어떻게 바꾸어야 할까?
7. 그레이디언트 부스팅 앙상블이 훈련 데이터에 과대적합되었다면 학습률을 높여야 할까요, 낮춰야 할까?

연습문제(2)

8. (3장에서 소개한) MNIST 데이터를 불러들여 훈련 세트, 검증 세트, 테스트 세트로 나누기(예를 들면 훈련에 50,000개 샘플, 검증에 10,000개 샘플, 테스트에 10,000개 샘플). 그런 다음 랜덤 포레스트 분류기, 엑스트라 트리 분류기, SVM 분류기 같은 여러 종류의 분류기를 훈련. 그리고 검증 세트에서 개개의 분류기보다 더 높은 성능을 내도록 이들을 간접 또는 직접 투표 방법을 사용해 앙상블로 연결해 보기. 앙상블을 얻고 나면 테스트 세트로 확인. 개개의 분류기와 비교해서 성능이 얼마나 향상되나?

9. 이전 연습문제의 각 분류기를 실행해서 검증 세트에서 예측을 만들고 그 결과로 새로운 훈련 세트를 만들어보기. 각 훈련 샘플은 하나의 이미지에 대한 전체 분류기의 예측을 담은 벡터이고 타깃은 이미지의 클래스. 새로운 훈련 세트에 분류기 하나를 훈련시켜보기.
   이제 테스트 세트에 앙상블을 평가. 테스트 세트의 각 이미지에 대해 모든 분류기로 예측을 만들고 앙상블의 예측 결과를 만들기 위해 블렌더에 그 예측을 주입. 앞서 만든 투표 분류기와 비교하면 어떤가?